Prompt:
Queremos definir la arquitectura para un sistema de paseadores de perros q vamos a hacer en .net. Quiero que me des las alternativas con sus pros y contras asi definimos la que mejor se adapta a nuestro proyecto.

https://docs.google.com/document/d/1CfkNpaKHhCiyo0wmKvaK5sJ0BycCtSv4/edit#heading=h.xi7ji74kx06k

en ese docs tenes la explicacion de como es el negocio, el modelo de dominio esta incompleto asique no le des importancia

Respuesta:
Analizando las reglas de negocio de su sistema (gestión de dueños y paseadores, búsqueda por criterios, validaciones de email, control de tarifas estrictamente positivas y trazabilidad de alta inmodificable), el proyecto tiene operaciones CRUD claras con reglas de validación precisas. Al ignorar el modelo de dominio inicial como pediste, podemos centrarnos puramente en cómo estructurar la solución en .NET.

Dado que están desarrollando en equipo (junto a Valentino y Fausto) y el proyecto requiere orden para evitar conflictos en GitHub, aquí tienen las principales alternativas de arquitectura para .NET, con sus pros y contras, para que puedan debatir cuál usar:

1. Arquitectura en N-Capas (N-Tier / Layered Architecture)

Es el enfoque clásico. Se divide el proyecto en tres capas técnicas principales: Presentación (API), Lógica de Negocio (Servicios) y Acceso a Datos (Repositorios con Entity Framework Core).

* **Pros:**
* **Baja curva de aprendizaje:** Es muy intuitiva y probablemente la que más se enseña al empezar con C# y .NET.
* **Rápida configuración:** Al ser un dominio relativamente pequeño (dueños, paseadores y búsquedas), pueden tener la estructura armada en un par de horas.
* **Separación básica:** Mantiene los controladores de la API limpios, delegando las reglas (como verificar que la tarifa sea > 0) a los servicios.

* **Contras:**
* **Acoplamiento:** La lógica de negocio tiende a depender fuertemente de cómo están modeladas las tablas en la base de datos.
* Puede llevar a la creación de "Servicios Gigantes" (ej. un `PaseadorService` con miles de líneas si el proyecto crece mucho).

2. Clean Architecture (Arquitectura Limpia / Cebolla)

Invierte las dependencias de la arquitectura clásica. El "Core" o "Dominio" está en el centro y no depende de nada (ni de Entity Framework, ni de la API). Todo lo demás (Base de datos, Web API) depende del Dominio.

* **Pros:**
* **Ideal para Reglas de Negocio:** Las reglas como "el email debe ser válido e irrepetible", "la tarifa no puede ser negativa" y "la fecha de alta es inmodificable" quedan encapsuladas perfectamente en el centro del sistema.
* **Altamente Testeable:** Es facilísimo escribir pruebas unitarias para validar las reglas sin necesidad de levantar una base de datos.
* **Práctica muy demandada:** Es el estándar de la industria hoy en día para proyectos serios en .NET. Suma mucho nivel al trabajo práctico.

* **Contras:**
* **Boilerplate (Código repetitivo):** Requiere crear múltiples proyectos dentro de la solución (Domain, Application, Infrastructure, Web) y definir muchas interfaces.
* **Sobrecarga inicial:** Para un sistema que al principio parece un CRUD, puede sentirse como "sobreingeniería" al principio.

3. Arquitectura de Cortes Verticales (Vertical Slice Architecture)

En lugar de organizar el código por capas técnicas (carpetas de Controllers, Services, Repositories), se organiza por "Funcionalidad" o "Caso de Uso" (ej. una carpeta `RegistrarPaseador`, otra `BuscarPaseadores`, otra `ModificarTarifa`). Suele implementarse junto con la librería **MediatR**.

* **Pros:**
* **Alta Cohesión:** Todo lo que se necesita para registrar un paseador (el endpoint, la lógica, el guardado en BD) está en un solo lugar.
* **Cero conflictos de Merge:** Como cada integrante del equipo trabaja en casos de uso distintos, casi nunca tocan los mismos archivos.
* **Evolución asimétrica:** Si la búsqueda de paseadores se vuelve muy compleja, se puede optimizar ese archivo sin afectar a cómo se registran los dueños.

* **Contras:**
* **Riesgo de código duplicado:** Al no haber servicios "compartidos", es fácil repetir lógica si no se sabe abstraer correctamente (por ejemplo, la validación del formato del email).
* Cambio de paradigma mental si vienen acostumbrados a las capas clásicas.

4. Arquitectura N-Capas con patrón CQRS

CQRS (Command Query Responsibility Segregation) separa estrictamente las operaciones que modifican estado (Commands: Registrar, Actualizar tarifa) de las que solo devuelven datos (Queries: Buscar paseador por zona).

* **Pros:**
* La función de "Buscar paseador", que probablemente sea la más consultada del sistema, se puede modelar de forma súper rápida leyendo directamente de la base de datos sin pasar por lógicas pesadas.
* Queda un código extremadamente ordenado y fácil de leer.

* **Contras:**
* Al igual que Clean Architecture, introduce complejidad y curvas de aprendizaje adicionales (casi siempre requiere usar MediatR y FluentValidation).

¿Cuál es la mejor recomendación para su proyecto?

Si el objetivo de la materia es **entregar un proyecto y asegurarse que funcione**, vayan por la **Arquitectura en N-Capas clásica**. Les va a permitir avanzar sin trabarse con conceptos avanzados de arquitectura.

Si buscan **excelencia técnica, sacar la mejor nota y llevarse un proyecto "de portfolio"**, les recomiendo **Clean Architecture** (pueden usar el template de Jason Taylor, que es muy famoso en la comunidad .NET). Este enfoque les asegurará que la inmutabilidad de la fecha de alta y las validaciones de las tarifas por hora queden blindadas dentro de sus entidades, independientemente de cómo guarden los datos.

Prompt:
Vamos a elegir la arquitectura en N.Capas clasica, que consejos nos das a la hora de arrancar el proyecto

Respuesta:
¡Excelente elección! La arquitectura en N-Capas es súper sólida, fácil de entender y les va a permitir avanzar rápido.
Para que el arranque sea organizado y no tengan dolores de cabeza (especialmente siendo tres trabajando en el mismo repositorio de GitHub), acá les dejo los mejores consejos paso a paso para estructurar el proyecto:

1. Estructura física de la Solución (Proyectos)

No metan todo en un solo proyecto. Creen una "Solución en Blanco" en Visual Studio (o por CLI) y agreguen tres proyectos separados. El flujo de dependencias (quién conoce a quién) debe ser estricto:

- 🐶 `Paseadores.Data` (Class Library): Acá va Entity Framework Core, el `DbContext` y los Repositorios. Conoce únicamente a la base de datos.
- 🐶 `Paseadores.Business` (Class Library): Acá van las interfaces y las clases de Servicio (ej. `PaseadorService`). **Esta capa debe tener una referencia a `Paseadores.Data`.**
- 🐶 `Paseadores.API` (Web API): Son los Controllers y el `Program.cs`. **Esta capa debe tener una referencia a `Paseadores.Business`.** *(Consejo clave: La API no debe hablar nunca directamente con Data, siempre pasa por Business).*

2. Usen DTOs (Data Transfer Objects) obligatoriamente

Nunca devuelvan ni reciban las entidades de la base de datos directamente en los Controllers de la API. Creen clases DTO en la capa `Business`.
Esto les soluciona directamente la regla de negocio de la **fecha de alta inmodificable**:

- Cuando alguien envía un `POST` para crear un paseador, recibe un `PaseadorCreateDTO` (que solo tiene Nombre, Apellido, Email, Zona, Tarifa).
- En el Servicio, ustedes instancian la entidad real `Paseador`, le copian esos datos y le asignan por código `FechaAlta = DateTime.UtcNow`. Como el DTO no tiene ese campo, el usuario jamás podrá manipular la fecha por error o malicia.

3. Centralicen las Reglas de Negocio en los Servicios

Sus reglas son muy claras (email único, tarifa > 0, datos obligatorios). Todo eso se valida en la capa `Business` antes de llamar al repositorio.

- *Recomendación pro:* En lugar de hacer mil `if (tarifa < 0)` a mano, instalen la librería **FluentValidation** en su proyecto Business. Les permite armar validaciones súper prolijas y limpias para sus DTOs.
- Para validar el **email único**, el Servicio debe inyectar el repositorio, hacer un `.AnyAsync(x => x.Email == nuevoEmail)` y si devuelve *true*, lanzar una excepción (ej. `InvalidOperationException` o una propia como `BusinessRuleException`) que la API atrape y devuelva como un HTTP 400 Bad Request.

4. Entity Framework Core y Base de Datos (Trabajo en equipo)

Como son tres programando al mismo tiempo, la base de datos puede ser un problema si cada uno tiene configuraciones distintas.

- **Consejo:** Usen Entity Framework Core con **Code-First** y Migraciones.
- **Base de datos:** Para desarrollo en equipo, **SQLite** es mágico. Te genera un archivo `.db` local en la carpeta del proyecto. Así nadie tiene que andar instalando SQL Server ni configurando contraseñas en sus máquinas. Cuando el proyecto esté listo, cambiarlo a SQL Server es literalmente tocar una línea de código.

5. Configuración de Inyección de Dependencias (Program.cs)

Recuerden que para que las capas se comuniquen, en el `Program.cs` de la API van a tener que registrar sus servicios y repositorios. Les quedará algo así:

```
C#
// En Program.cs
builder.Services.AddScoped<IDuenoRepository, DuenoRepository>();
builder.Services.AddScoped<IDuenoService, DuenoService>();
builder.Services.AddScoped<IPaseadorRepository, PaseadorRepository>();
```

builder.Services.AddScoped<IPaseadorService, PaseadorService>();

6. Organización en GitHub (Git Flow Básico)

Como van a estar tocando código Fausto, Valentino y vos, **no trabajen directo sobre la rama** main.

1. Creen ramas por funcionalidad (ej: feature/crud-duenos, feature/validacion-tarifas).
2. Cada uno trabaja en su rama.
3. Cuando terminan, hacen un *Pull Request* (PR) hacia main y se revisan el código entre ustedes antes de unirlo. Esto evita que se pisen el trabajo o rompan lo que hizo el otro.

Para arrancar hoy:

Abran la terminal o Visual Studio, creen la solución, agreguen los 3 proyectos, referéncienlos, instalen Entity Framework Core en la capa Data, y armen su primer controlador (ej. DueñoController) que solo devuelva un "Hola Mundo" para comprobar que todo compila bien. ¡Éxitos con ese TP!

Conclusión

Luego de utilizar la Inteligencia Artificial para explorar y analizar las distintas alternativas arquitectónicas para nuestro Sistema de Paseadores de Perros, el equipo debatió las opciones presentadas y decidimos avanzar con la **Arquitectura en N-Capas**.

Si bien la IA nos presentó alternativas como *Clean Architecture* o *Vertical Slices*, llegamos a la conclusión de que la arquitectura en capas tradicionales es la que mejor se adapta a nuestro contexto actual como estudiantes y a los objetivos de la materia.

Nuestra principal comparativa fue contra la **Clean Architecture**. Sabemos que esta última es el estándar actual de la industria para proyectos complejos y que encapsula de manera excelente las reglas de negocio. Sin embargo, Clean Architecture introduce una gran cantidad de abstracciones, interfaces y código repetitivo (*boilerplate*) que añade una curva de aprendizaje y una sobrecarga inicial muy alta.

Al ser este uno de nuestros primeros proyectos integrales de la facultad, consideramos que implementar una arquitectura tan compleja podría confundirnos un poco. Preferimos optar por un enfoque más simple y directo como el de N-Capas (Presentación, Lógica de Negocio y Acceso a Datos), que nos permite cumplir con el principio de separación de responsabilidades, pero manteniendo un flujo de trabajo intuitivo.

Elegir N-Capas nos viene perfecto en esta etapa académica para **terminar de asentar los conocimientos fundacionales del desarrollo backend**. Queremos concentrarnos en entender a la perfección cómo se diseña y expone una Web API, cómo armar correctamente los endpoints, cómo gestionar las inyecciones de dependencias, y cómo realizar las conexiones y *queries* a la base de datos utilizando Entity Framework Core.

En resumen, la Arquitectura en N-Capas nos brinda el equilibrio exacto que necesitamos hoy: es lo suficientemente estructurada para trabajar en equipo mediante GitHub sin generar un código espagueti, y lo suficientemente directa para permitirnos afianzar los conceptos técnicos clave sin perdernos en la sobreingeniería de arquitecturas más avanzadas.